Web Scraping using Python

What is Web Scraping?

- Web Scraping is a technique to extract a large amount of data from several websites. The term "**scraping**" refers to obtaining the information from another source (webpages) and saving it into a local file.

What is Web Scraping?

🗣 Web Scraping is a technique to extract a large amount of data from several websites. The term "**scraping**" refers to obtaining the information from another source (webpages) and saving it into a local file.

Web scraping

- **Web scraping** is the process of collecting and parsing raw data from the Web.
- Many disciplines, such as data science, business intelligence, and investigative reporting, can benefit enormously from collecting and analyzing data from

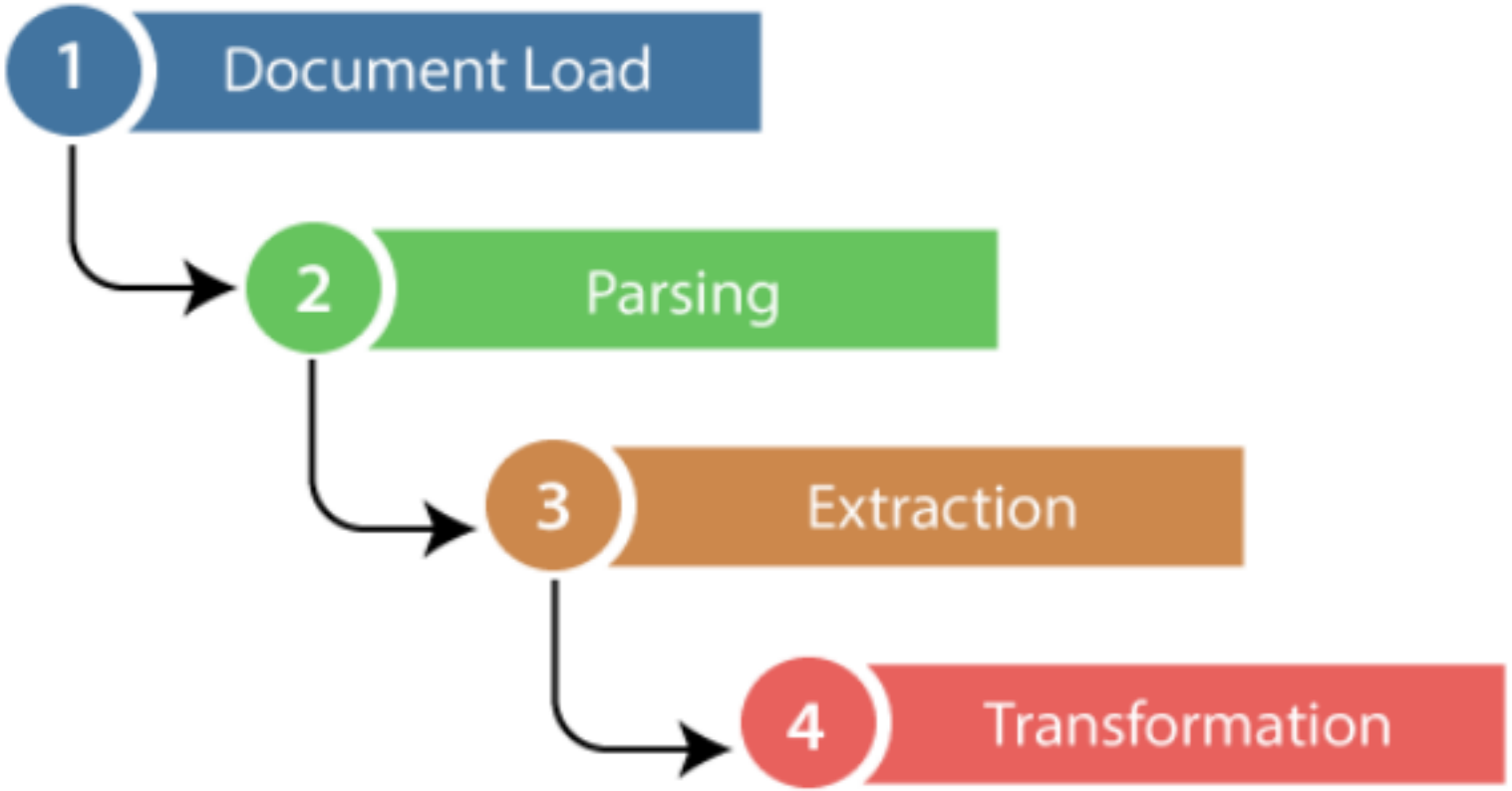
websites.

usage of web scrapping

- **Dynamic Price Monitoring:-**It is widely used to collect data from several online shopping sites and compare the prices of products and make profitable pricing decisions.
- **Market Research:-**provides the data with a guaranteed level of reliability and accuracy.
- **Email Gathering:-**
- Many companies use personals e-mail data for email marketing. They can target the specific audience for their marketing.
- **News and Content Monitoring:-**scraping provides the ultimate solution to monitoring and parsing the most critical stories.
- **Social Media Scrapping:-**Web Scrapping plays an essential role in extracting data from social media websites such as **Twitter, Facebook,** and **Instagram,** to find the trending topics.
- **Research and Development:-**The large set of data such as **general information, statistics, and temperature** is scrapped from

websites, which is analyzed and used to carry out surveys or research and development.

Steps to web Scraping



Scrape and Parse Text From Websites

- Collecting data from websites using an automated process is known as web scraping.

- **Important:** Before using your Python skills for web scraping, you should always check your target website's acceptable use policy to see if accessing the website with automated tools is a violation of its terms of use. Legally, web scraping against the wishes of a website is very much a gray area.

Can you scrape from all the

websites?

- Scraping makes the website traffic spike and may cause the breakdown of the website server. Thus, not all websites allow people to scrape. How do you know which websites are allowed or not? You can look at the '**robots.txt**' file of the website. You just simply put robots.txt after the URL that you want to scrape and you will see information on whether the website host allows you to scrape the website.
- Ex: https://www.google.com/robots.txt
- https://www.flipkart.com/robots.txt

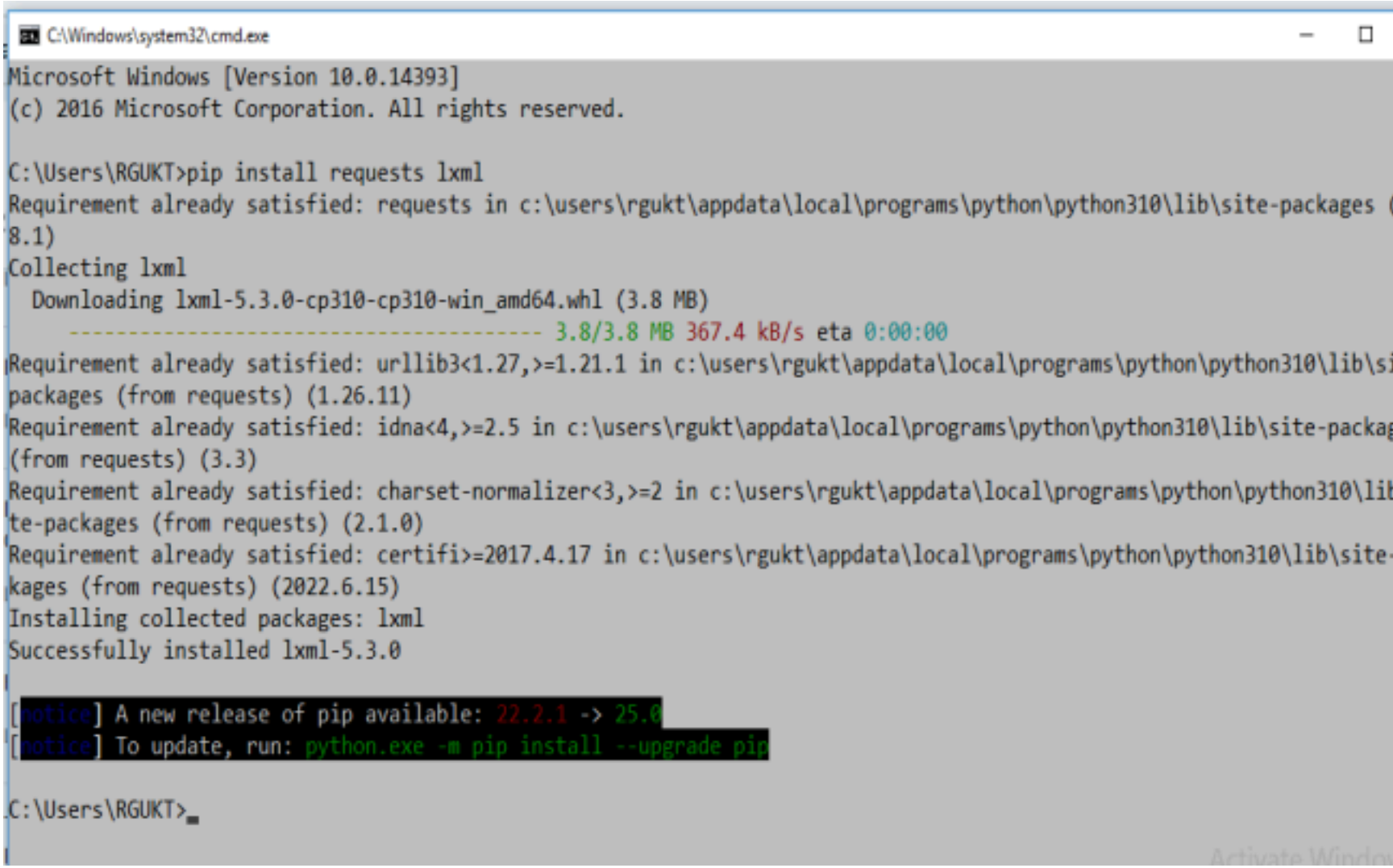
Essential Packages and Tools for Python Web Scraping

- The latest version of Python , offers a rich set of tools and libraries specifically designed for web scraping, making it easier than ever to retrieve data from the web efficiently and effectively.
- Requests Module:Requests is a simple and elegant HTTP library for Python that makes HTTP requests. Due to its ease of use, it is often the starting point for web scraping.
- BeautifulSoup Library:Beautiful Soup is a library for parsing HTML and XML documents. It creates parse trees from page source code that can easily extract data.
- Selenium
- Lxml
- Urllib Module:urllib3 is a powerful, user-friendly HTTP client for Python. It builds on the standard library's urllib module and adds many features.

MechanicalSoup :MechanicalSoup is a library for automating interaction with websites, built on top of the BeautifulSoup library and the Requests library.

- Scrapy :Scrapy is an open-source and collaborative web crawling framework for Python. It is used for large-scale web scraping tasks.

Installing lxml library



Library used for web scrapping

- Python has a vast collection of libraries and also provides a very useful library for web scrapping.
- The most commonly used library for web scraping in Python is **Beautiful Soup, Requests, and Selenium**.
- **Beautiful Soup:** It helps you parse the HTML or XML documents into a readable format. It allows you to search different elements within the documents and help you retrieve required information faster.
- **Requests:** It is a Python module in which you can send HTTP requests to retrieve contents. It helps you to access website HTML contents or API by sending Get or Post requests.
- **Selenium:** It is widely used for website testing and it allows you to automate different events(clicking, scrolling, etc) on the website to get the results you want.
- Note:**Selenium is preferred if you need to interact with the website(JavaScript events) and if not I will prefer Requests + Beautiful Soup because it's faster and easier**.

Your First Web Scraper

- One useful package for web scraping that you can find in Python's standard library is urllib, which contains tools for working with URLs.
- In particular, the urllib.request module contains a function called urlopen() that can be used to open a URL within a program.

Example:

- from urllib.request import urlopen
- The web page that we'll open is at the following URL: url = " https://www.flipkart.com/
- To open the web page, pass url to

urlopen(): page = urlopen(url)

- To extract the HTML from the page, first use the HTTPResponse object's .**read()** method, which returns a sequence of bytes. Then use .**decode()** to decode the bytes to a string using UTF-8:

Cont….

- >>> html_bytes = page.read()
- >>> html = html_bytes.decode("utf-8")
  - Once you have the HTML as text, you can extract information from it in a couple of different ways.

- **Note:** Web scraping in Python or any other

language can be tedious. No two websites are organized the same way, and HTML is often messy. Moreover, websites change over time. Web scrapers that work today are not guaranteed to work next year—or next week, for that matter!

Extract Text From HTML With String Methods

- you can use .find() to search through the text of the HTML for the <title> tags and extract the title of the web page.
- If you know the index of the first character of the title and the first character of the closing </title> tag, then you can use a string slice to extract the title.
- you can get the index of the opening <title> tag by passing the string "<title>" to .find():
- Now get the index of the closing </title> tag by passing the

string "</title>" to .find():

- Finally, you can extract the title by slicing the html string:

Try extracting the title from this new URL using (example)

- >>> url =“ https://www.flipkart.com/”
- >>> page = urlopen(url)
- >>> html = page.read().decode("utf-8")
- >>> start_index = html.find("<title>") + len("<title>")
- >>> end_index = html.find("</title>")
- >>> title = html[start_index:end_index]
- >>> title

Requests Module

- The requests library is used for making HTTP

requests to a specific URL and returns the response. Python requests provide inbuilt functionalities for managing both the request and response.

- Python requests module has several built-in methods to make HTTP requests to specified URI using GET, POST, PUT, PATCH, or HEAD requests. A HTTP request is meant to either retrieve data from a specified URI or to push data to a server. It works as a request-response protocol between a client and a server. Here we will be using the GET request. The GET method is used to retrieve information from the given server using a given URI. The GET method sends the encoded user information appended to the page request.

Example

- **Importing Libraries:** The code imports the requests

library for making HTTP requests and the BeautifulSoup class from the bs4 library for parsing HTML.

- **Making a GET Request:** It sends a GET request to 'https://www.geeksforgeeks.org/python-programming language/' and stores the response in the variable r.
- **Checking Status Code:** It prints the status code of the response, typically 200 for success.
- **Parsing the HTML** : The HTML content of the response is parsed using BeautifulSoup and stored in the variable soup.
- **Printing the Prettified HTML:** It prints the prettified version of the parsed HTML content for readability and analysis.

Example

- import requests
- from bs4 import BeautifulSoup

🗣 # Making a GET request
🗣 r = requests.get(https://example.com’)

🗣 # check status code for response
received 🗣 # success code - 200
🗣 print(r)
🗣 # Parsing the HTML
🗣 soup = BeautifulSoup(r.content,
'html.parser') 🗣 print(soup.prettify())
🗣 s = soup.find('div', class_='entry-content')
🗣 content = soup.find_all('p')
🗣 print(content)

Extract Text From HTML With Regular Expressions

🗣 import re
🗣 from urllib.request import urlopen

- url = "http://www.flipkart.com"
- page = urlopen(url) html = page.read().decode("utf-8")
- pattern = "<title.*?>.*?</title.*?>“
- match results = re.search(pattern, html, re.IGNORECASE)
- title = match_results.group()
- title = re.sub("<.*?>", "", title) # Remove HTML tags
- print(title)

Create a BeautifulSoup Object

- from bs4 import BeautifulSoup
- from urllib.request import urlopen
- url =

"http://olympus.realpython.org/profiles/dionysus
" page = urlopen(url)

- html = page.read().decode("utf-8")
- soup = BeautifulSoup(html, "html.parser")

Cont…

- soup.find_all("a") returns a list of all links in the HTML source.
- You can loop over this list to print out all the links on the webpage:
- for link in soup.find_all("a"):
- link_url = base_url + link["href"]
- print(link_url)

Web Scraping Example : Scraping Flipkart Website

- Pre-requisites:
- Python 2.x or Python 3.x with **Selenium**, **BeautifulSoup**, **pandas** libraries installed
- Google-chrome browser

Step 1: Find the URL that you want to scrape

- For this example, we are going scrape **Flipkart** website to extract the Price, Name, and Rating of Laptops. The URL for this page is

https://www.flipkart.com/laptops/~buyback guarantee-on-laptops-/pr?sid=6bo%2Cb5g&uniqBStoreParam1=val1&wid=11. productCard.PMU_V2.

Step 2: Inspecting the Page

🗣 The data is usually nested in tags. So, we inspect the page to see, under which tag the data we want to scrape is nested. To inspect the page, just right click on the element and click on “Inspect”.

- **Step 3: Find the data you want to extract**Let's extract the Price, Name, and Rating which is in the "div" tag respectively.
- **Step 4: Write the code**
- First, let's create a Python file.
- First, let us import all the necessary libraries:

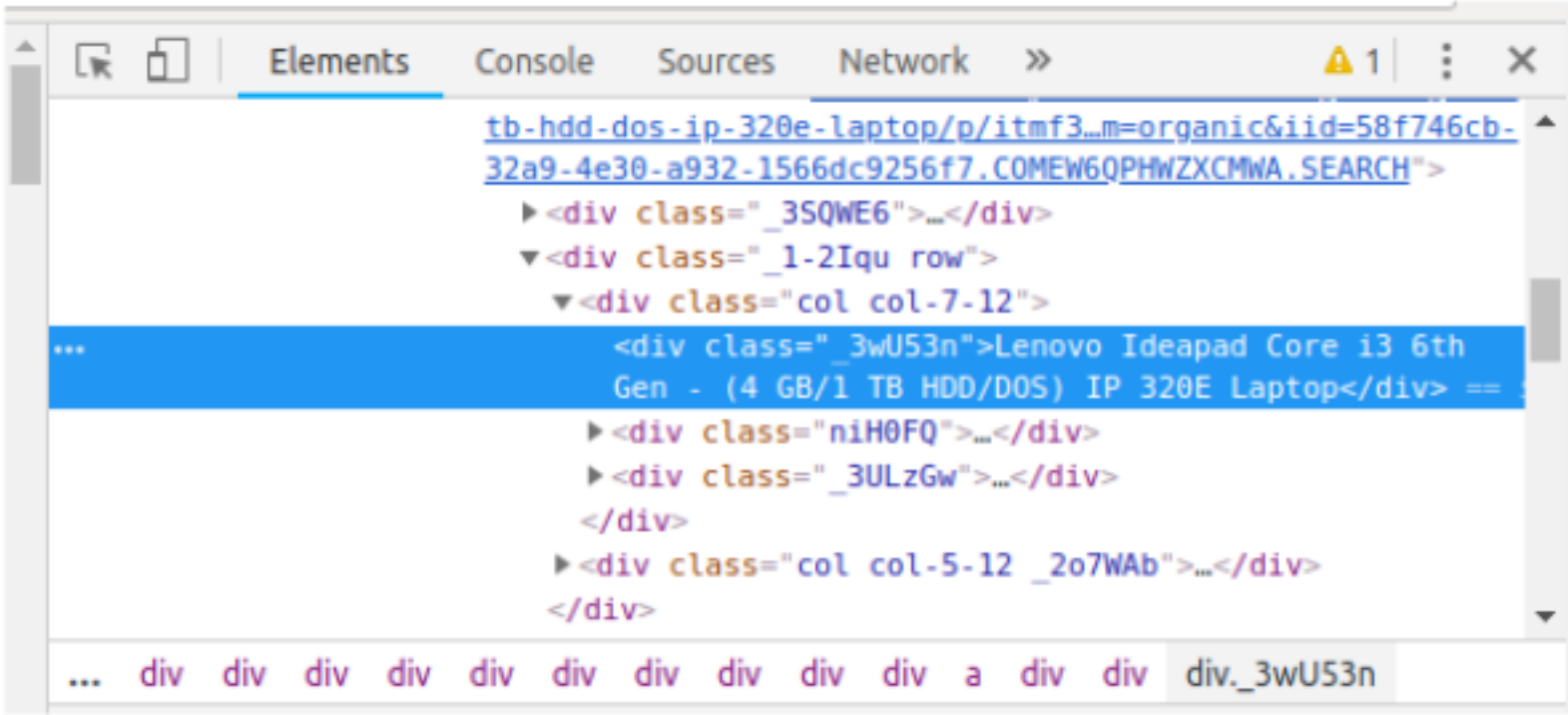
To configure web driver to use Chrome browser, we have to set the path to chrome driver

🗣 I will find the div tags with those respective class names, extract the data and store the data in a variable.

🗣 **Step 5: Run the code and extract the data** 🗣 **Step 6: Store the data in a required format**

Cont..

🗣 When you click on the “Inspect” tab, you will see a “Browser Inspector Box” open.


Elements
Console
Sources
Network
1
tb-hdd-dos-ip-320e-laptop/p/itmf3…m=organic&iid=58f746cb-32a9-4e30-a932-1566dc9256f7.COMEW6QPHWZXCMWA.SEARCH">
…

Lenovo Ideapad Core i3 6th Gen - (4 GB/1 TB HDD/DOS) IP 320E Laptop ==
…
…

…

... div div div div div div div div div div a div div div._3wU53n